UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESTRUCTURA DE DATOS
CATEDRÁTICO: LUIS FERNANDO ESPINO BARRIOS
TUTOR ACADÉMICO: RUBÉN ALEJANDRO RALDA MEJÍA



MANUAL DE USUARIO
JAVALANG INTERPRETER

OTTONIEL FABRICIO VÁSQUEZ PINEDA
CARNÉ: 202307820
SECCIÓN: B

GUATEMALA, 25 DE OCTUBRE DEL 2,025

1. Introducción

JavaLang Interpreter es una herramienta educativa diseñada para interpretar programas escritos en el lenguaje JavaLang, una variante académica con sintaxis similar a Java. Esta herramienta incluye una interfaz gráfica que permite editar código, ejecutarlo y visualizar reportes de análisis (AST, tabla de símbolos y errores).

2. Instalación

Requisitos del sistema:

- Sistema operativo Linux
- Compilador GCC
- Herramientas Flex y Bison
- Biblioteca GTK3
- Graphviz (para visualización del AST)

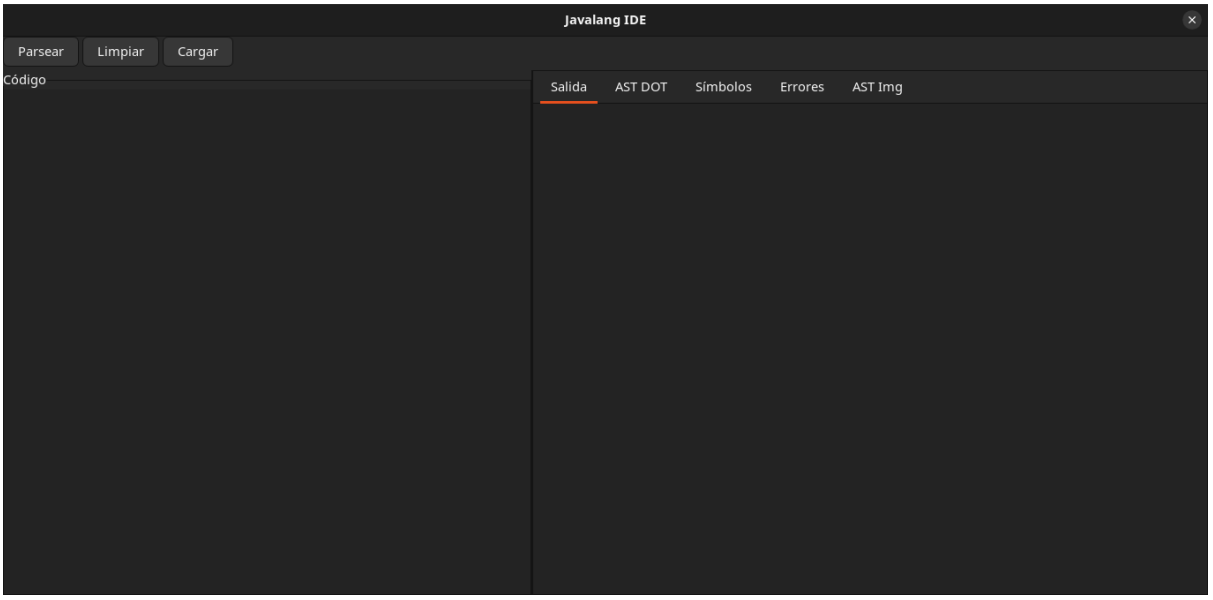
Pasos de instalación:

1. Actualizar lista de paquetes
   sudo apt update
2. instalación de herramientas
   sudo apt install flex bison

3. instalar GTK3 y las dependencias de desarrollo
   sudo apt install libgtj-3.dev

4. instalar Graphviz
   sudo apt install graphviz
5. Instalar herramientas adicionales
   sudo apt install git make cmake
6. Clonar el repositorio del proyecto:
   git clone https://github.com/Farot30/OLC2_2s2025_202307820
   cd OLC2_2S2025_202307820
7. Compilar proyecto
   make
8. Ejecutar la aplicación:
   ./run_gui.sh

3. Uso de la herramienta



La interfaz contiene las siguietnes áreas:

- Editor de código: para escribir código Javalang
- Panel de pestañas:
    - Salida de consola
    - Código DOT del AST
    - Tabla de símbolos
    - Reporte de errores
    - Visualización del AST como imagen.

Crear y Editar Código:

Haga clic en el editor de código.

Escriba su programa en lenguaje JavaLang

Ejecutar código:

1. Cargue un archivo existente con el botón “cargar” (archivo con extensión .usl).
2. Escriba código directamente en el editor.
3. Presione el botón “Parsear” para:
    - Analizar el código léxicamente.

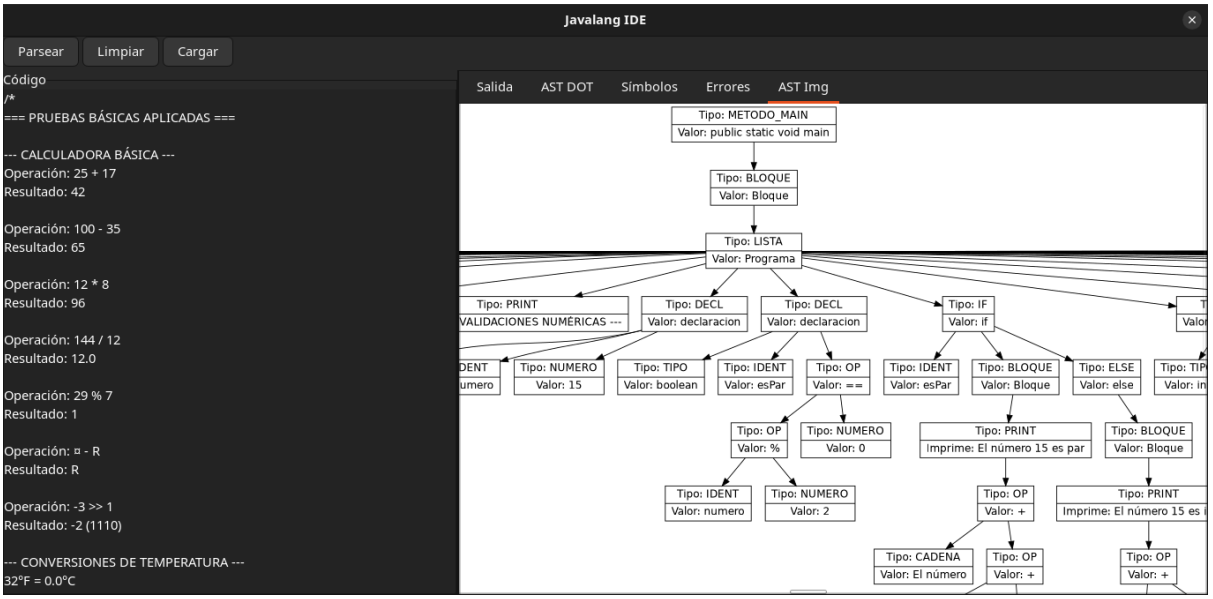
- Analizar sintácticamente.
- Realizar análisis semántico
- Generar todos los reportes

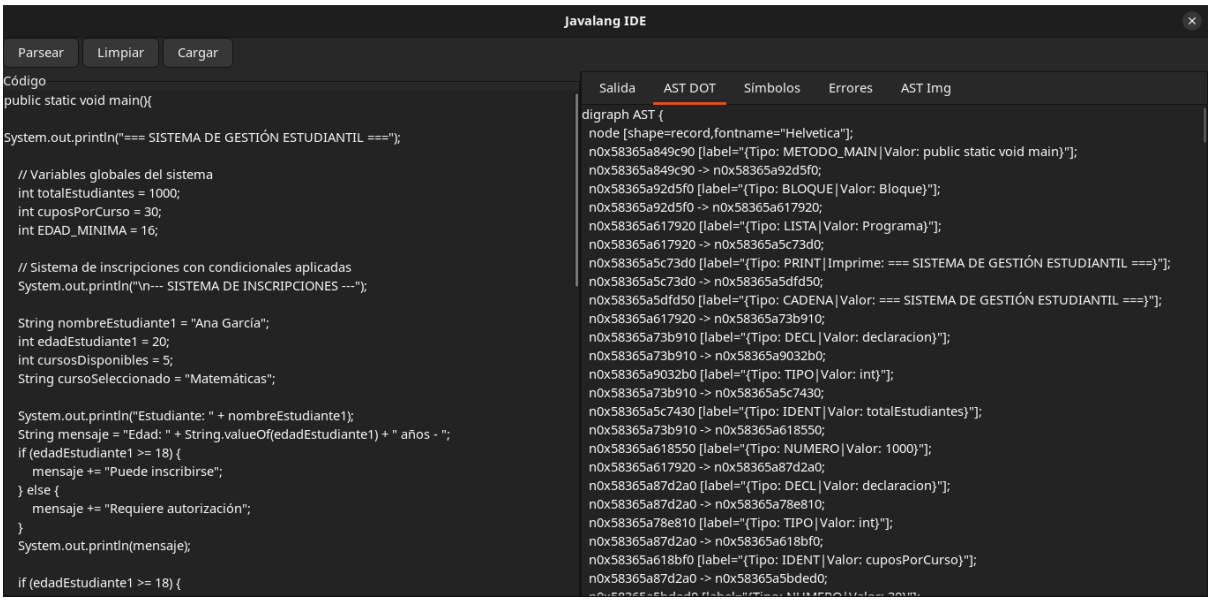
Ejemplo de código válido:

```
public static void main(){
    int x = 8;
    System.out.println(x);
}
```

Interpretación de Reportes

Reporte de AST:

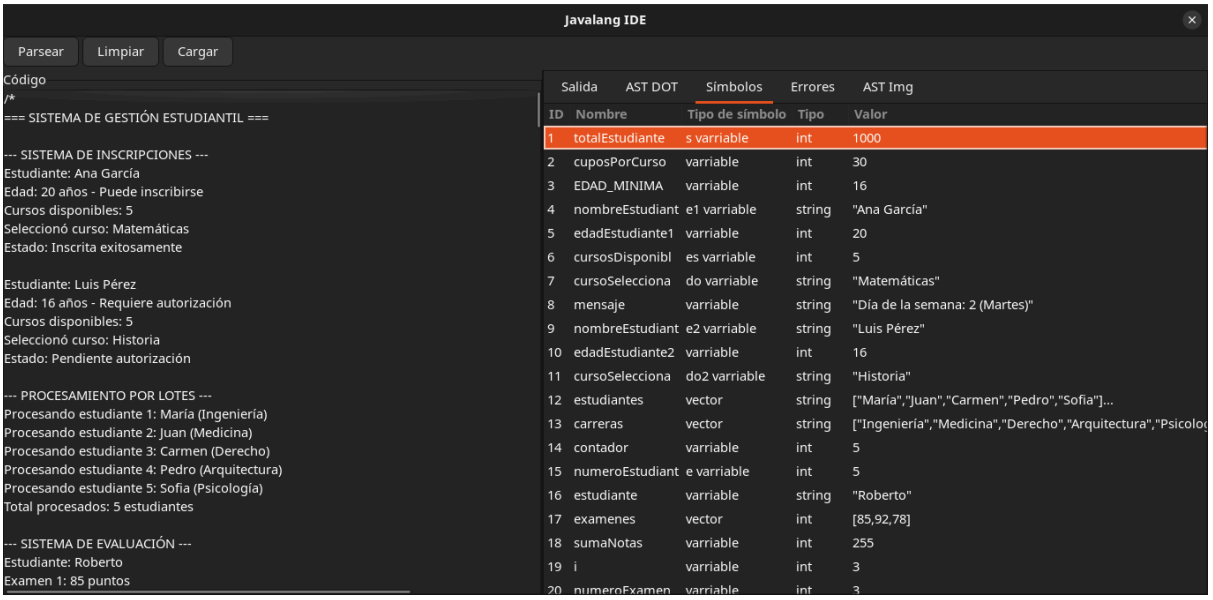




El AST (Abstract Syntax Tree) muestra la estructura jerárquica del código analizado. Se presenta en dos formatos:

1. Código Dot: Representación textual para expertos.
2. Imagen: Visualización gráfica generada con Graphviz

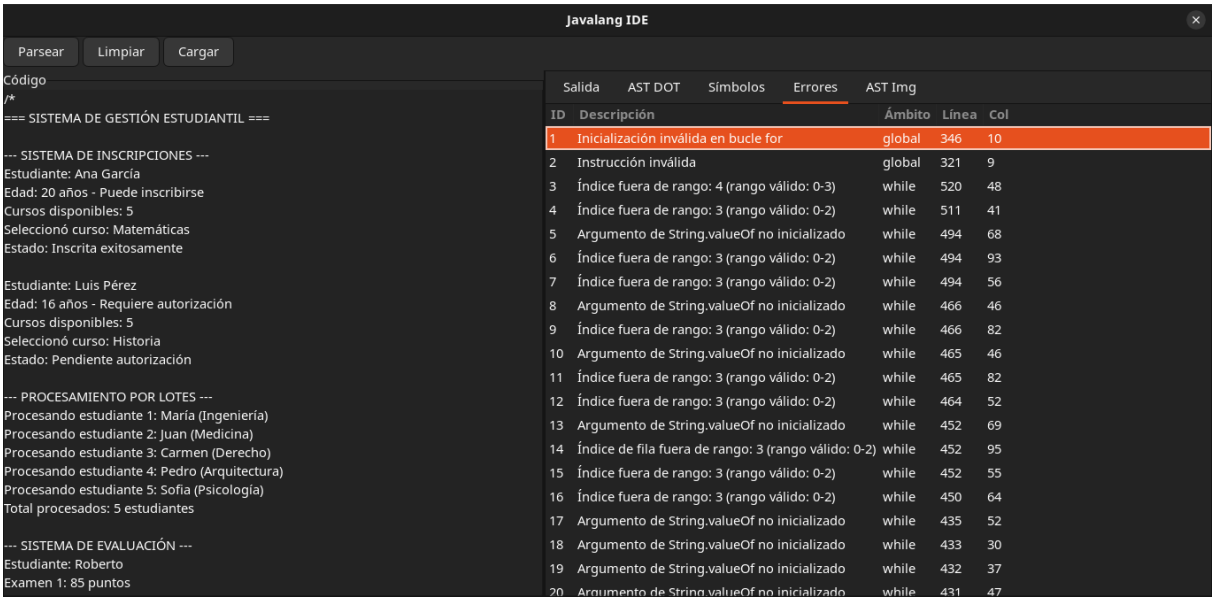
Tabla de Símbolos:



Muestra información detallada sobre:

- Variables declaradas (ID, nombre, tipo, valor, ámbito, línea, columna)
- Funciones definidas
- Tipos de datos utilizados
- Ámbitos de visibilidad

Reporte de Errores

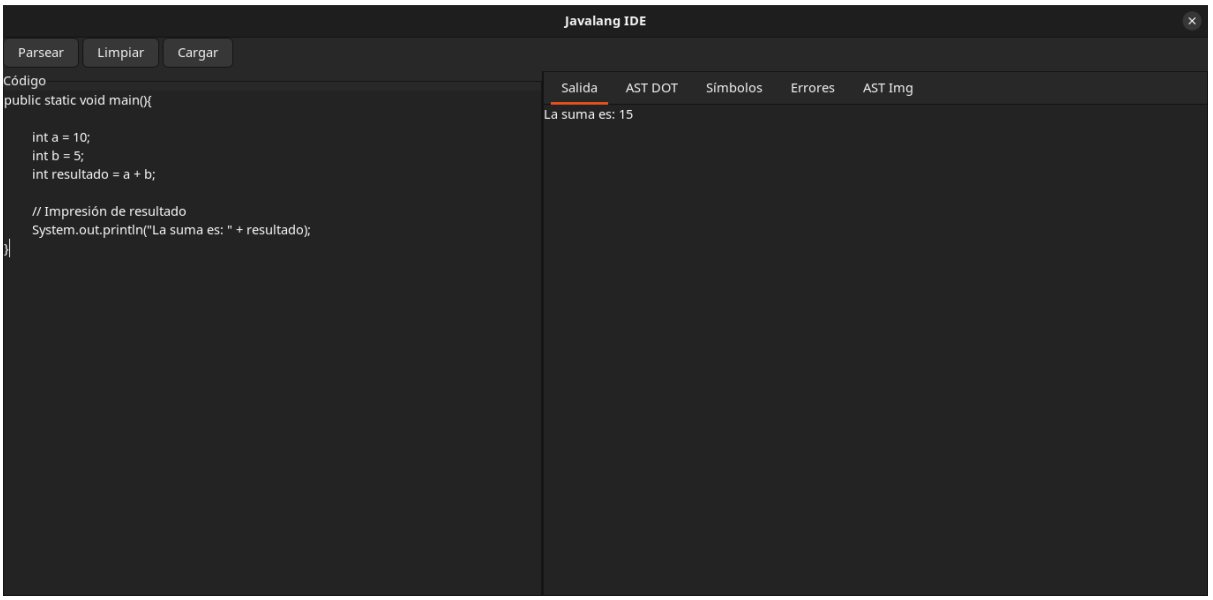


Clasifica los errores encontrados durante el análisis:

- Errores léxicos: Caracteres no válidos
- Errores sintácticos: Estructuras incorrectas
- Errores Semánticos: Tipos incompatibles, variables no declaradas

Ejemplos de Sesión de Uso

Ejemplo 1:

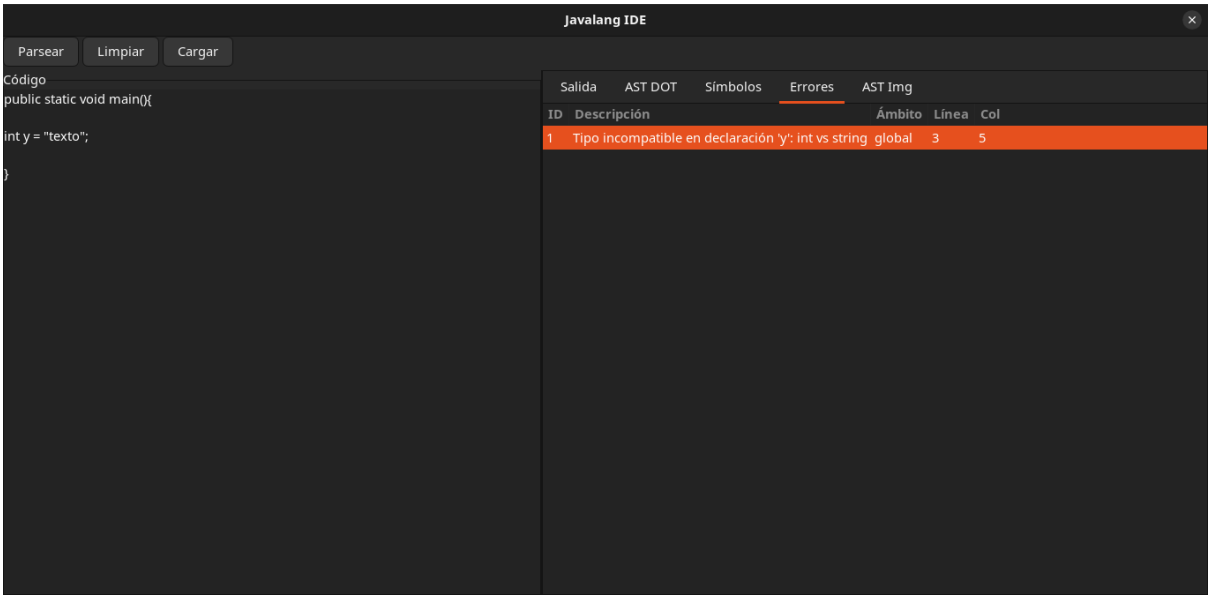



```
public static void main(){
        int a = 10;
        int b = 5;
        int resultado = a + b;
        // Impresión de resultado
        System.out.println("La suma es: " + resultado);
}
```

Resultado esperado:

- Salida en consola: "La suma es: 15"
- AST generado correctamente
- Tabla de símbolos con variables a, b, resultado
- Sin errores reportados

Ejemplo 2: Programa con Error semántico



Código de error

```
public static void main(){
        int y = "texto";
}
```

Resultado esperado

- Reporte de error semántico mostrado
- Mensaje: "No se puede asignar String a variable int"
- Tabla de símbolos muestra variable con error

Creación de archivo.s

Ahora se creará un archivo.s en la cual tendrá el contenido de código arm64 generado al apachar el boton de parser

```
.global _start
.extern parse_int
.extern parse_double
.extern parse_float
.extern valuesOf_int
.extern valueOf_bool
.extern valueOf_float
.extern valueOf_double
.extern valueOf_long
.extern printf


// declaraciones de variables
.data
format_int_0: .asciz "%d\n"
posicionCodigo: .word -1
codigoBuscado: .word 1003
codigosProducto: .word 0, 0, 0, 0, 0

// constantes
.section .rodata

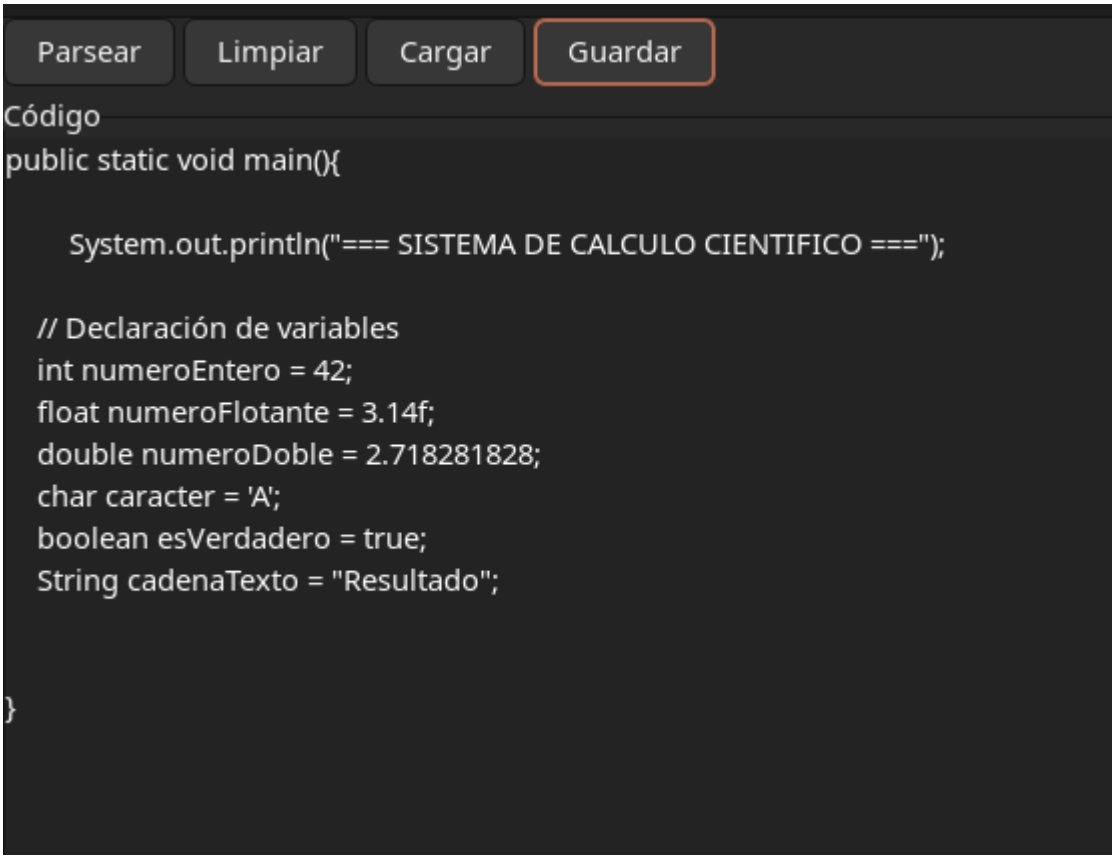
// seccion bss
.bss

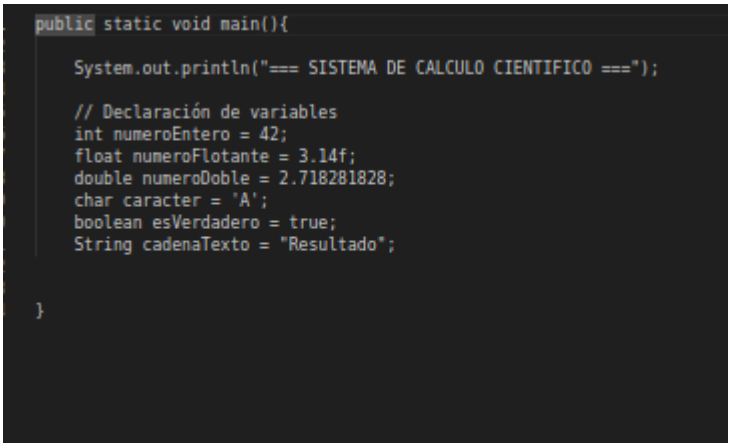
// código ejecutable
.text

_start:
    // Punto de entrada del programa
    // Escritura en vector: codigosProducto[índice] = valor
    mov w1, #0
    mov w2, #1001
    ldr x3, =codigosProducto           // Dirección base del vector
    str w2, [x3, w1, SXTW #2]  // Almacenar valor
    // Escritura en vector: codigosProducto[índice] = valor
    mov w1, #1
    mov w2, #1002
    ldr x3, =codigosProducto           // Dirección base del vector
    str w2, [x3, w1, SXTW #2]  // Almacenar valor
    // Escritura en vector: codigosProducto[índice] = valor
    mov w1, #2
    mov w2, #1003
    ldr x3, =codigosProducto           // Dirección base del vector
    str w2, [x3, w1, SXTW #2]  // Almacenar valor
    // Escritura en vector: codigosProducto[índice] = valor
    mov w1, #3
    mov w2, #1004
    ldr x3, =codigosProducto           // Dirección base del vector
    str w2, [x3, w1, SXTW #2]  // Almacenar valor
    // Escritura en vector: codigosProducto[índice] = valor
```

Botón guardar

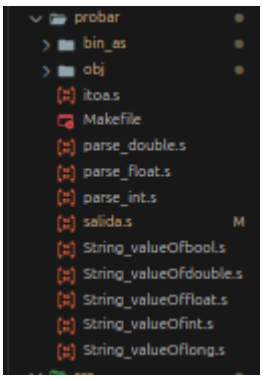
Este botón guardará el contenido de entrada en un archivo con extensión .usl





Ejecutar código ARM64

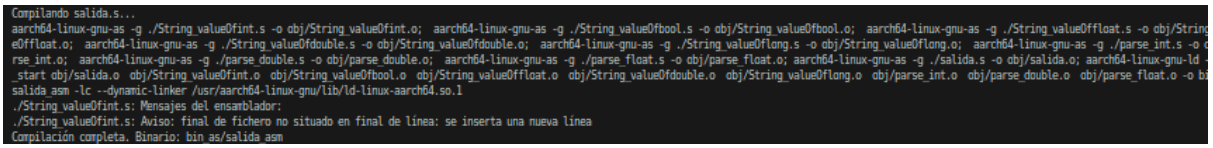
Para poder ejecutar el código arm64, tiene que entrar a una carpeta llamada probar, en la cual ahí se podrá comprobar si en verdad funciona el código arm64.



El siguiente paso es abrir la terminal dentro de la carpeta probar, en la cual al momento de abrirla podrá utilizar un comando que es el siguiente.

make archivo=salida.s

Este comando servirá para poder compilar el código arm64 en qemu y hacer verificaciones para que no haya ningún error al compilar el programa, cuando se sabe que se ejecutó bien, puede que le muestre algo parecido a esto.



Si se compilo bien el archivo, entonces podrá poner el siguiente comando que sería:

make run

Este comando le servirá para ejecutar ya el código arm64 en qemu y ver el resultado que le puede dar que sería como el siguiente

